ChatGPT

Executive Summary

We propose a **cloud-native, microservice-based onboarding and KYC platform** for a Mutual Fund app. This system lets new investors self-register, submit and verify documents (PAN, Aadhaar), and complete *electronic KYC* (eKYC) via Aadhaar/DigiLocker or manual upload. The process is streamlined, fully digital, and compliant with RBI/SEBI norms. We will support **JWT-based authentication**, secure file uploads (to cloud storage), OCR data extraction, and optional facial verification. An admin console tracks pending KYC, and audit logs capture every action for compliance【9†L89-L97】【21†L153-L161】. Notifications (email/SMS) keep users informed. Our architecture leverages AWS services (or equivalents) for scaling, security, and cost-efficiency.

Key highlights:

- **Paperless eKYC**: Users authorize UIDAI to share Aadhaar data, enabling a fast, fraud-resistant KYC【9†L89-L97】. If unavailable, manual PAN/Aadhaar upload with OCR is supported.
- **User-friendly flow**: A "Ready to Start" page guides even non-technical users through requirements. The frontend (e.g. React) uses clear labels and step-by-step prompts.
- **Secure backend**: Microservices (Node.js/Python/Java) handle Auth, Profile, KYC, Doc-processing, etc. JWT tokens secure APIs. Data is encrypted in transit and at rest.
- **Compliance & Audit**: Every action (upload, verification, admin decision) is logged. Audit trails are chronological records ("who did what when") that satisfy regulators【20†L50-L58】【21†L64-L72】.
- **Scalable DevOps**: Dockerized services deploy via CI/CD (GitHub Actions/Jenkins) into AWS (EKS/ECS/Lambda). We use managed DBs (RDS/MongoDB), S3 for files, and AWS Textract or Google Vision for OCR. IAM roles, KMS encryption, and VPC isolation ensure security.

This document details the problem, solution approach, personas, user journey, data flows (with Mermaid diagrams), API and DB designs, tech stack choices, and a step-by-step development plan (checklists, testing, runbook).

Problem Statement

Financial regulators (RBI, SEBI) *require* stringent KYC to prevent fraud and money laundering. Traditional KYC (paper forms, bank visits, and offline verification) is slow, error-prone, and costly【9†L77-L85】. In the digital age, investors expect *instant, online account opening*. Legacy onboarding (monolithic apps, batch processing) can take days to weeks. To stay competitive, a mutual fund app must provide a *seamless, end-to-end digital onboarding* that is (a) **compliant**, (b) **quick**, and (c) **user-friendly**.

Challenges include: verifying identity (PAN/Aadhaar), extracting data from various documents, checking liveness for selfies, and maintaining detailed logs for audits【21†L153-L161】. We must design a system that guides an **average user** (even 8th grade literacy) through these steps, while also giving admins tools to review or override as needed.

Goals and Objectives

1. **Paperless Onboarding:** Enable 100% digital sign-up and KYC (no manual paperwork). Use Aadhaar-based e-KYC and digital signatures to avoid printing/forms【9†L89-L97】.
2. **Fast User Experience:** Target under 5 minutes to complete onboarding, similar to Groww's "completely online and paperless" process【14†L70-L78】. Show clear instructions at each step.
3. **Strong Security:** Use JWT for auth, TLS for data in transit, encryption at rest (KMS). Protect PII and financial data.
4. **Compliance & Audit:** Log every event (uploads, verification results, approvals) with user and timestamp. Meet KYC audit requirements (e.g. retain records ≥5 years【21†L104-L113】).
5. **Modular Architecture:** Build with microservices so we can independently scale Auth, KYC, OCR, etc. Use cloud services (AWS preferred) for reliability.
6. **Tech Flexibility:** Recommend robust, industry-standard tools (React, Node.js, MongoDB, AWS Textract, Rekognition) but note alternatives.
7. **Developer Clarity:** Provide a detailed implementation guide so an engineering team (and even a tech-savvy layperson) can follow each step of building and deploying the system.

User Personas and Stories

- **Rahul (First-time Investor, Age 24):** Tech-savvy university student. "I want to sign up for a mutual fund app quickly using my Aadhaar and PAN. I expect to do it on my phone with minimal fuss."
- **Priya (Traditional Investor, Age 50):** Not very tech-oriented. "I have never opened an account online. I need clear guidance and feedback after each step. If I get stuck, I want the app to explain in simple language what I need."
- **Anita (KYC Officer, Admin):** Works at the mutual fund company. "I need to review users whose documents failed automated checks. The dashboard should list pending KYC tasks and let me approve or reject with comments."
- **Dev Team:** Needs precise API specs, tech stack decisions, and deployment plans to build the system end-to-end.

From these personas, user stories include:

1. **User Registration/Login:** As *Rahul*, I want to register using email/mobile and password, so I can create an account.
2. **Profile Entry:** As *Rahul*, I want to enter or auto-fill my personal details (name, address, income), so the KYC form is completed accurately.
3. **Document Upload:** As *Rahul*, I want to upload my PAN and Aadhaar images (or fetch via DigiLocker), so that my identity and address proofs are verified.
4. **Digital KYC:** As *Rahul*, I want to do an Aadhaar e-sign (OTP-based), so I can electronically sign the account opening form.
5. **Face Verification:** As *Rahul*, I want the app to capture a selfie and verify it matches my ID photo, so I know my account is secure.
6. **Admin Review:** As *Anita*, I want to see a list of KYC submissions with status, so I can process any that need manual attention.
7. **Notifications:** As *Rahul*, I want SMS/email updates on my KYC status (approved, pending, or rejected), so I know when I can start investing.

8. **Audit Transparency:** As *Priya*, I (or my manager) want a log of every step of my onboarding, so I have assurance of privacy and accountability.

End-to-End Onboarding Journey

Below is a simplified user journey from landing to final approved dashboard:

1. **Preview/Readiness Page**: User is shown a friendly introduction: *"Welcome! To complete your account opening, you'll need your PAN card, Aadhaar linked to your mobile for OTP, and a bank account. This takes about 5 minutes."* (see **UX Copy** below).

2. **Registration**: User enters email/mobile and password (or phone OTP). We validate input and create a user record. Send a verification link/OTP for account activation. On success, issue a JWT for login.

3. **Login**: User logs in with email/mobile and password. On valid credentials, the backend returns a JWT (with expiration). The frontend stores it (e.g. in secure HttpOnly cookie or memory) for authorization.

4. **Profile Details**: Once logged in, the user fills out personal details: full name, date of birth, gender, income, occupation, parents' names, etc. Some fields (like name, DOB) can be pre-filled if coming from Aadhaar e-KYC or user profile. This saves later KYC form filling.

5. **KYC Option**: User chooses KYC method:

6. **Digital KYC (DigiLocker)**: If user clicks “Use DigiLocker”, the app initiates OAuth with DigiLocker (Govt. of India). The user logs into DigiLocker (via Aadhaar OTP) and consents to share documents (like DigiLocker’s PAN, Aadhaar). Our backend then calls DigiLocker APIs to retrieve signed XML or PDFs of these IDs【1†L101-L110】【9†L89-L97】.

7. **Manual Upload**: Else, user is prompted to upload images (or PDFs) of required documents: PAN card, Aadhaar card (front/back). Each upload has a clear “take picture or choose file” UI.

8. **OCR & Data Extraction**: Uploaded images are stored in S3 (or cloud storage). An OCR service (AWS Textract or Google Vision) reads text from these images. We auto-parse name, DOB, PAN number, and Aadhaar number from the IDs. The parsed data pre-fills the KYC form. (See **Tech Stack**: AWS Textract excels at forms/fields【16†L946-L951】.) The user verifies/corrects the extracted data before proceeding.

9. **Aadhaar e-Sign (Optional)**: If PAN and Aadhaar data are captured, we ask the user to electronically sign the KYC form. This uses UIDAI’s eSign API: the user enters their Aadhaar number, receives an OTP, and signs. Upon success, a digitally signed consent/Account Opening Form (AOF) is generated. This step is what Groww does (“E-SIGN AOF” via Aadhaar OTP【14†L110-L119】).

10. **Face Capture and Liveness**: The app then asks the user to take a selfie (or short video). A **Face Recognition** service compares this selfie to the photo extracted from Aadhaar (or PAN). We can use AWS Rekognition or Azure Face API. Liveness detection (blink or challenge) ensures it's not a spoof. If the match and liveness pass, we mark biometric verification success.

11. **Submit for Verification**: With documents and biometrics done, the user submits the application. The backend updates the KYC request status to “Pending Verification”. If any document failed OCR or face-match, we flag “Needs Review”.

12. **Admin Review**: An admin portal (protected) shows a queue of pending KYC applications. Admin (role-based) can click a user, view the uploaded docs, OCR text, and face-match result. They can approve or reject with comments. For rejections, we notify the user to retry or contact support.

13. **Status Notifications**: Throughout the process, send timely updates:

    - On registration and login successes.
    - On submission of KYC documents.
    - On final approval or rejection of KYC (via email/SMS using a service like AWS SNS or Twilio).
    - These notifications keep the user informed and engaged.

14. **Audit Trail**: Behind the scenes, every action (user sign-up, login, uploads, verification results, admin decisions) is logged in an audit database. This log can be queried during compliance audits to show the “chain of custody” for each user’s data【21†L153-L161】.

15. **Investor Dashboard**: Once KYC is approved, the user can access the main app: investing in mutual funds, tracking SIPs, etc. The dashboard shows a welcome message and their account ID. Until then, they may see a “Verification in Progress” status page.

Throughout, the UX is simplified (no jargon, step labels like Step 1 of 5, clear errors). Even an 8th-grader could follow prompts like “Upload your PAN card here” or “Enter OTP sent to your phone.”

Dataflow Diagrams (Mermaid)

Below are some **Mermaid** flowcharts illustrating key processes.

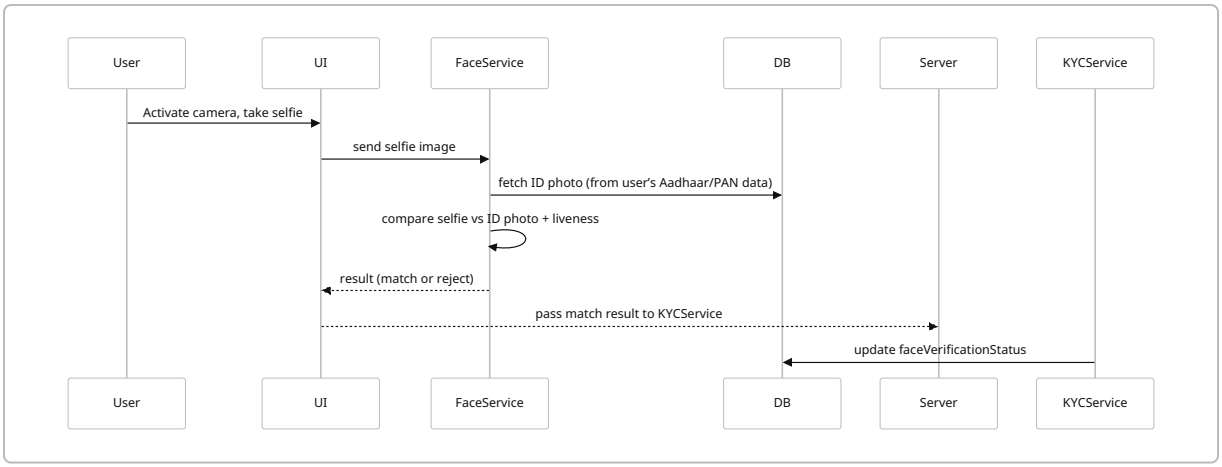
User Registration & Login Flow:

```
flowchart LR
  A[Landing Page] --> B[Register / Login]
  B -->|Register| C{New User?}
  C -->|Yes| D[Send Verification OTP/Email]
  D --> E[Confirm & Create Account]
  E --> F[Issue JWT Token]
  C -->|No (Login)| G[Enter Credentials]
  G --> H[Verify + Issue JWT Token]
  F --> I[Redirect to Profile Setup]
  H --> I
```

KYC Document Submission Flow:

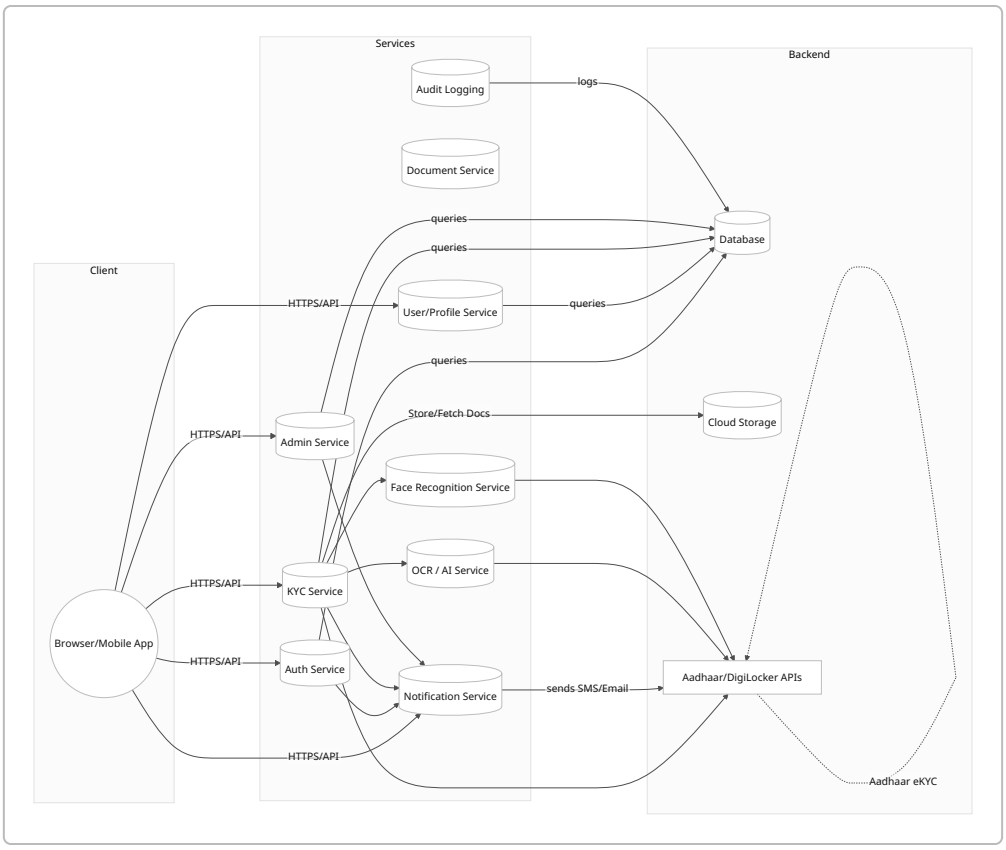
```
flowchart TD
  U[User] --> X[Frontend KYC Form]
  X --> Y{Choose KYC Method}
  Y -->|DigiLocker| D[Initiate DigiLocker OAuth]
  Y -->|Manual| M[Show Upload Fields]
  D --> DL[DigiLocker Login/Consent]
  DL -->|Success| E[Backend calls DigiLocker API]
  E --> DocDB[Save DigiLocker Docs to Storage]
  M -->|Upload| DocDB
  DocDB --> OCR[Trigger OCR Service]
  OCR --> Parsed[Parsed Data (Name, etc.)]
  Parsed --> X[Auto-fill Form]
  X -->|User Clicks Submit| K[Submit KYC Request]
  K --> DB[Update KYC Status: Pending]
```

Facial Verification Flow:



High-Level Architecture Diagram

A microservices approach simplifies scaling and maintenance. We recommend **AWS (or equivalent)** for infrastructure. Below is a high-level component diagram.



Components Responsibilities:

- **API Gateway / Frontend**: Handles HTTPS requests. May be a load balancer or AWS API Gateway. For simplicity, frontend (React/Vue) calls backend services directly.
- **Auth Service**: Manages user registration, login (password hash, tokens), JWT issuance, refresh tokens.
- **User/Profile Service**: Stores user demographic data (name, address, occupation). Integrates with Auth Service.
- **KYC Service**: Orchestrates the KYC process: receives document uploads, calls OCR, triggers DigiLocker, records status. Maintains a "KycRequest" for each user.
- **Document Service**: Handles file uploads/downloads. Stores files in S3 (or GCP/Azure storage), keeps metadata (URL, doc type) in DB.
- **OCR Service**: A microservice or serverless function that invokes an OCR engine (AWS Textract or Google Vision) on stored documents and returns parsed fields.
- **Face Recognition Service**: Compares user selfie with ID photo using a vision API (e.g. AWS Rekognition). Also checks liveness.
- **Admin Service**: Web UI/API for internal admin use. Lists pending KYC, shows user data and docs, enables approve/reject actions.
- **Notification Service**: Sends emails/SMS (via AWS SES/SNS or Twilio). E.g., "Your KYC is approved" message.
- **Audit Logging**: Centralized logging of events (maybe stored in DB or a log system). Tracks user actions, admin decisions, etc. (fulfills regulatory logging【20†L50-L58】).

Each service runs in a container (Docker) or serverless function, with well-defined APIs. They communicate mostly via REST/JSON or message queues for async tasks (e.g. SNS topics or SQS queues).

API Contracts (Examples)

Below are sample REST endpoints. All data is JSON over HTTPS, secured by JWT in Authorization headers (except /login, /register).

- **POST /api/auth/register**
  Request: `{ "email": "...", "mobile": "...", "password": "..." }`
  Response: `201 Created` or error. (Sends verification email/OTP.)

- **POST /api/auth/login**
  Request: `{ "username": "...", "password": "..." }`
  Response: `200 OK` with `{ "token": "...", "user": { id, email, role } }` if success, `401` if invalid.

- **GET /api/user/profile** (Protected)
  Response: `200 OK` with `{ "name": "...", "dob": "...", "gender": "...", ... }`.

- **POST /api/user/profile** (Protected)
  Request: `{ "name": "...", "dob": "...", "gender": "...", "income": ... }`
  Response: `200 OK`.

- **POST /api/kyc/upload** (Protected, form-data)
  Request: multipart/form-data with fields `type` (e.g. "PAN", "AADHAAR") and file.
  Response: `201 Created`, body `{ "docId": "...", "ocrData": {...} }`.

- **GET /api/kyc/status** (Protected)
  Response: `200 OK` `{ "kycStatus": "Pending", "faceMatch": "Success", ... }`.

- **POST /api/kyc/verify-aadhaar** (Protected)
  Initiates Aadhaar eSign; request body `{ "aadhaarNumber": "...", "otp": "..." }`.
  Response: `200 OK` or error.

- **POST /api/face/verify** (Protected)
  Request: `{ "selfieImageId": "..." }`.
  Response: `200 OK` `{ "matched": true, "confidence": 98.7 }`.

- **GET /api/admin/kyc/pending** (Admin only)
  Response: `200 OK` array of pending KYC requests with user info.

- **POST /api/admin/kyc/approve** (Admin only)
  Request: `{ "userId": "...", "remarks": "..." }`.
  Response: `200 OK` (updates status to “Approved”).

- **POST /api/admin/kyc/reject** (Admin only)
  Request: `{ "userId": "...", "reason": "ID not clear" }`.
  Response: `200 OK` (status "Rejected").

Note: In all responses, include useful fields and error messages. The API names above are examples; design for RESTful consistency (e.g. /api/users/me, /api/kyc, /api/docs).

Data Models / Database Schema

We can use a NoSQL DB (like MongoDB) or relational (Postgres). Below is a simplified schema (as MongoDB collections):

Collection	Fields (example)	Description
users	`_id (UUID)`, `email`, `mobile`, `passwordHash`, `role`, `createdAt`, `emailVerified`	Auth credentials and basic user.
profiles	`_id (UUID)`, `userId (FK)`, `name`, `dob`, `gender`, `income`, `occupation`, `address`	Additional personal details for KYC.
kyc_requests	`_id`, `userId`, `pan`, `aadhaar`, `status` (`Pending/Approved/Rejected`), `submittedAt`, `reviewedAt`, `remarks`, `faceMatch`	Tracks each user's KYC process status.
documents	`_id`, `userId`, `type` (`PAN/AADHAAR/SIGNATURE`), `url`, `uploadTime`, `ocrData` (JSON), `verificationStatus`	Uploaded doc metadata, URLs (S3 links), OCR output.
notifications	`_id`, `userId`, `type`, `message`, `sentAt`, `read`	Email/SMS logs for user.
audit_logs	`_id`, `userId` (nullable), `action`, `timestamp`, `details`	Compliance log: e.g. "LOGIN", "UPLOAD_PAN", "ADMIN_APPROVED" with context.

Sample Document Record (in MongoDB JSON style):

```
{
  "_id": "doc123",
  "userId": "user456",
  "type": "PAN",
  "url": "https://s3.amazonaws.com/bucket/doc123.jpg",
  "uploadTime": "2026-06-23T12:34:56Z",
  "ocrData": {
    "PAN": "ABCDE1234F",
```

```
    "Name": "John Doe",
    "FatherName": "Richard Roe",
    "DOB": "1980-01-01"
  },
  "verificationStatus": "Pending"
}
```

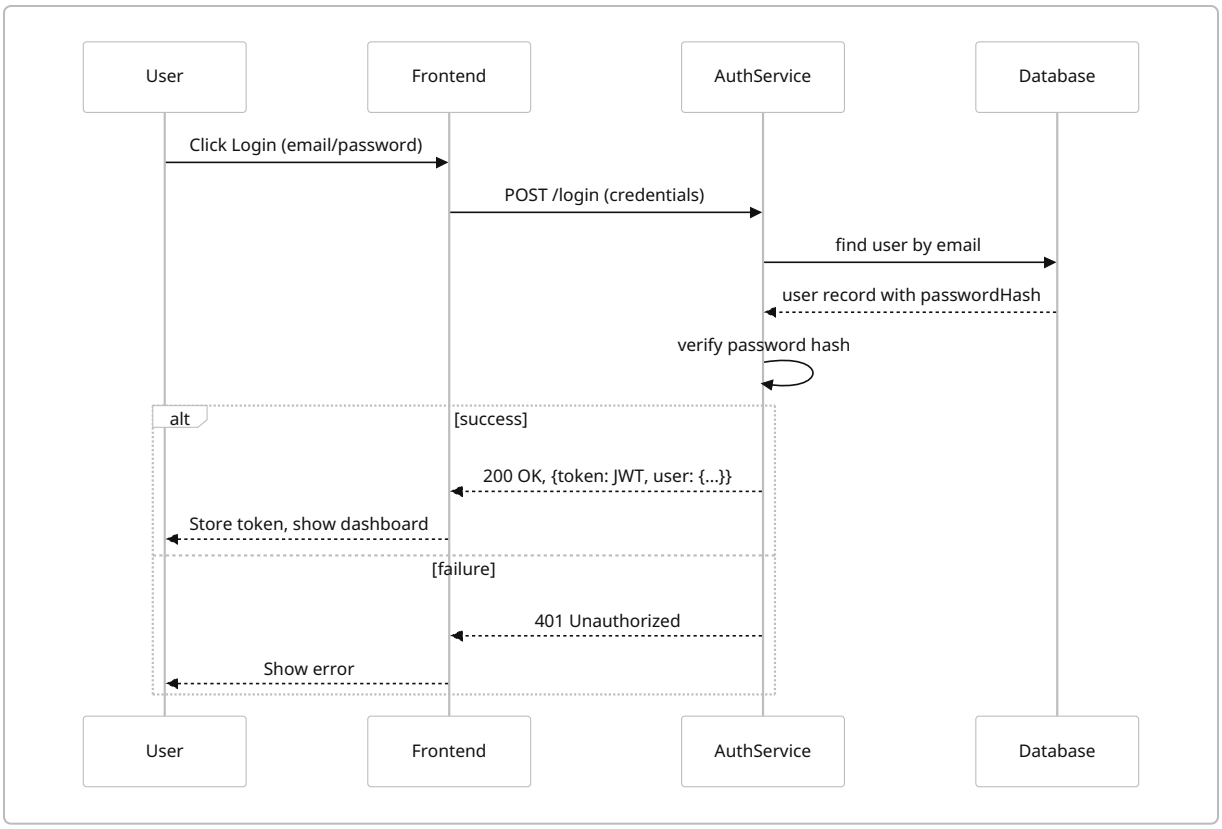
Sample JWT Payload:
When the user logs in, we issue a JWT payload like:

```
{
  "userId": "user456",
  "email": "john.doe@example.com",
  "role": "user",
  "iat": 1687635300,
  "exp": 1687642500
}
```

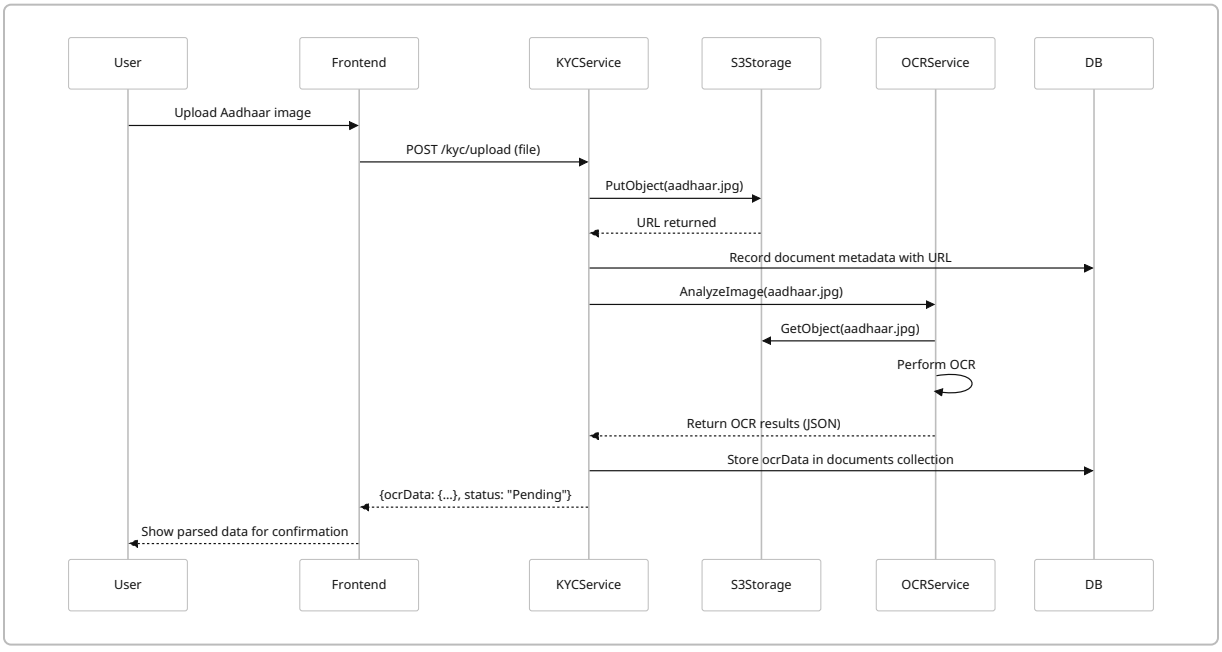
This token (signed with a strong secret or RSA key) is sent in `Authorization: Bearer <token>` on future requests.

Sequence Diagrams

Login and Token Issuance:



Document Upload and OCR:



DigiLocker Flow (Sketch):

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant K as KYCService
    participant DL as DigiLocker API
    U->>F: Click "Fetch from DigiLocker"
    F->>DL: Redirect for DigiLocker OAuth (user logs in, consents)
    DL-->>F: Returns OAuth token
    F->>K: GET /kyc/digilocker (with user token)
    K->>DL: GET /dlpartner/issuer/list (DigiLocker API using our client
credentials and user OTP)
    DL-->>K: list of documents (PAN, Aadhaar, etc.)
    K->>DL: GET /dlpartner/issuer/download?docid=PAN_ID
    DL-->>K: returns PAN PDF (digitally signed)
    K->>S: upload PAN PDF to S3
    K->>O: OCR on PAN PDF (or use DL's JSON if available)
    ... repeat for Aadhaar ...
    K->>DB: save docs and extracted data
    K-->>F: {docsFetched: true, panData: {...}, aadhaarData: {...}}
    F-->>U: Show data for review
```

(Implementation note: DigiLocker APIs use OAuth flow and download endpoints. We handle token exchange securely on backend【1†L101-L110】.)

Technical Stack and Tools

We recommend a modern, widely used tech stack. Below are primary choices and alternatives:

- **Frontend:** *React.js* (or *Vue.js*/Angular). React is popular, component-based, and has many UI libraries (Material UI, Ant Design). It can call backend APIs and handle JWT token storage.
  Alternative: React Native (for mobile app), or a hybrid framework. But as web is main, React is a safe choice.

- **Backend Language/Framework:** *Node.js* with Express (JavaScript/TypeScript) is fast for I/O and has JWT libraries.
  Alternatives: Python (Flask/FastAPI) or Java (Spring Boot) or Go. All can serve REST APIs, but Node.js is common in fintech stacks and fits JSON well.

- **Database:** *MongoDB* (NoSQL) for flexibility in storing semi-structured KYC data (OCR JSON, audit logs). It scales horizontally.
  Alternative: PostgreSQL or MySQL (relational). RDBMS enforces schemas but is also mature. Either is fine. For fast development, MongoDB's document model suits our variable "documents" structure.

- **File/Document Storage:** *AWS S3*. It's cheap, durable, and easy to serve via pre-signed URLs. We store user-uploaded docs and any images in S3.
  Alternative: Google Cloud Storage or Azure Blob. Self-hosting (minIO) is possible but more ops.

- **OCR Service:** *AWS Textract* or *Google Cloud Vision*. Textract is designed for forms and tables (like PAN cards), whereas Vision API is strong at free-text and general images.
  We might use Textract to extract structured fields from IDs (it can recognize PAN card layout), with fallback to Google Vision if needed. IronSoftware notes Textract is "precise" with forms【16†L946-L951】.
  Alternative: On-prem Tesseract OCR (open source) could be used, but often less accurate and harder to scale. Given no cost constraints, managed cloud OCR is preferred for accuracy and speed.

- **Face/Liveness Verification:** *AWS Rekognition* or *Azure Face API*. Both provide face match (compare source and target photos) and basic liveness detection. Rekognition is easy to integrate on AWS.
  Alternative: Using open-source libraries (OpenCV with Dlib) plus a challenge (blink detection). But that's complex to implement robustly. Third-party vendors like Face++ are also options. For simplicity, AWS Rekognition is recommended.

- **Messaging/Events:** *AWS SQS/SNS* for decoupled messaging (e.g. sending a KYCCompleted event to trigger notifications). Kafka (Amazon MSK) is possible for high throughput if needed【27†L1-L4】. Given moderate scale, SNS+SQS suffices.

- **Notifications (SMS/Email):** *AWS SNS+SES* or services like Twilio (SMS) and SendGrid (email). SES is cost-effective for email. For SMS in India, integrate with providers like MSG91 or AWS Pinpoint.

- **CI/CD:** *GitHub Actions* or *Jenkins*. Set up pipelines to lint, test, build Docker images, and deploy to AWS. For AWS, use CodePipeline/CodeBuild or Terraform for infra as code.

- **Containerization/Orchestration:** *Docker* for all services. Deploy on *AWS EKS* (Kubernetes) or *ECS/Fargate*. EKS offers flexibility; ECS is simpler.
  Alternative: Serverless (AWS Lambda) for stateless parts (e.g. OCR invocation, simple APIs) can reduce ops but requires re-architecture (stateless functions). For an MVP, containers on ECS/EKS is straightforward.

- **Infrastructure:** AWS (preferred).

- VPC with public/private subnets.
- RDS (Postgres/MySQL) or DocumentDB (Mongo-compatible) for DB.
- S3 buckets for docs.
- Amazon Cognito (optional) for user management and JWT generation.
- AWS Rekognition, Textract, Lambda, MSK, SQS, SNS, CloudWatch.

Alternative Clouds: GCP or Azure equivalents: GKE/AKS, Cloud SQL/Firestore, Cloud Vision, Azure Face. But AWS is asked (via hint) for recommended.

- **Monitoring:** *Prometheus/Grafana* for metrics, or AWS CloudWatch (with custom metrics, dashboards). Use OpenTelemetry or AWS X-Ray for tracing.

Logging: Centralize logs with ELK Stack or AWS CloudWatch Logs. Ensure audit log data is immutable (write-once storage or WORM policy if needed).

- **Security:** TLS (HTTPS) everywhere. JWT secrets stored in *AWS Secrets Manager*. Encrypt DB volumes and S3 with KMS. Use *IAM Roles* for least privilege (each service only accesses its own DB/S3). Regular security scans of dependencies and container images. Compliance with data privacy laws (GDPR/India's DPDP expected soon) – allow user data deletion and encryption.

DevOps, Security, and Scalability

- **Containerization & Orchestration:** Package each microservice in Docker. Use an orchestration platform (AWS EKS/ECS). For example, deploy Auth, KYC, Admin services as separate Kubernetes Deployments with appropriate CPU/memory. Use Horizontal Pod Autoscaler to scale out when CPU/memory thresholds are hit.

- **CI/CD Pipeline:**

- Commit triggers workflow:

1. Run unit tests (Jest/PyTest).
2. Build Docker images and push to ECR (AWS container registry).
3. Deploy to staging (via Helm or CloudFormation).
4. Run integration tests (API tests via Postman/Newman).
5. On success, trigger production deployment (with manual approval).
   Use Git branching (feature branches, PR reviews, main branch for deploy). Infrastructure changes managed with Terraform or CloudFormation (in code repo).

- **Infrastructure (AWS)**:

- VPC: public subnets (load balancers) and private subnets (services, DB).
- API Gateway/ALB fronts services.
- EKS cluster with namespaces per service.
- DB: Amazon RDS (for SQL) or Amazon DocumentDB/Mongo Atlas, with multi-AZ for HA.
- S3 bucket with bucket policy: TLS only and least-privileged IAM role for KYCService to upload.
- **Encryption**: Enable `encryption-at-rest` on RDS (KMS key), S3 default encryption. All secrets (DB credentials, JWT keys, OAuth keys) in AWS Secrets Manager or Parameter Store (encrypted).
- **IAM**: Fine-grained roles: e.g. AuthService role can query `users` table; KYCService role can read/write `kyc_requests` and put to S3; OCRService role calls Textract; Rekognition role has face compare permission; Admin role has read/write on everything.

- **Networking Security**: Use security groups (only allow HTTP on port 443 from internet to ALB). Private subnets for internal services, accessible only via VPC. Use AWS WAF on API Gateway to block common web attacks.

- **Monitoring & Logging:**

- Collect logs from all services to CloudWatch Logs or an ELK cluster. Use structured JSON logs (winston/Bunyan for Node, similar for others).
- Metrics: Track request latency, error rates, CPU/memory, number of users, KYC throughput. Use CloudWatch Alarms (or Prometheus) to alert on anomalies (e.g. high error rate or drop in onboarding rate).

- Audit Trail: Store in DB or secure log store. Regularly backup logs and set retention (e.g. 7 years to cover any regulatory requirement).

- **Scaling & Cost:**

- Use AWS Auto Scaling Groups or EKS auto-scaler. For unpredictable load (e.g. month-end SIP rush), ensure there is headroom.
- Choose *spot instances* for non-critical workers (e.g. OCR) to save cost, fall back to on-demand if needed.
- Leverage *serverless* where possible: e.g. AWS Lambda for short tasks (sending notifications, updating logs). Lambda has pay-per-use, which is cheap when idle.
- For user sessions, JWT means stateless servers (no sticky sessions needed).
- AWS Free Tier / Savings Plans can lower costs. Track usage (S3 storage, Lambda invocations, Rekognition calls have costs).
- Plan multi-AZ deployments for fault tolerance. Use RDS read replicas if needed.

- Ensure backups: Daily DB snapshots, S3 versioning if needed.

- **Security/Compliance:**

- We will follow standards like OWASP Top 10. Input validation on all endpoints (no SQL injection, etc.).
- Use **JWT expiring tokens** and a refresh token mechanism for user sessions. Invalidate tokens on logout (track in DB or short expiration).
- Encrypt PII (Aadhaar number, etc.) in DB columns. Tokenize sensitive info if needed.
- Comply with KYC retention rules: retain all submitted docs and logs for at least 5 years【21†L104-L113】.
- Implement data purge policies (delete user account on request, in line with privacy laws).
- Regular security audits and penetration tests.

Alternative Approaches

- **Manual Upload Only:** Simplest to implement. Users upload PDFs/images of PAN/Aadhaar, we OCR them. Pros: Doesn't rely on DigiLocker. Cons: More user effort, higher error rate in OCR, potential fraud risk. Requires photo ID signature and wet signature courier (like some brokers do). Modern UX demands fully digital KYC, so this is fallback only.

- **DigiLocker eKYC:** Government-backed, free service where users log in via Aadhaar OTP【1†L101-L110】. The system can fetch digitally signed copies of Aadhaar/PAN. This is secure and user-friendly for those with Aadhaar-linked mobile. *Pros:* Very fast, no document scanning needed, high trust. *Cons:* Requires user registration with DigiLocker and Aadhaar-mobile link. We should offer both DigiLocker and manual.

Reference: “DigiLocker allows users to sign in/up with Aadhaar/OTP... verified documents can be shared with providers【1†L81-L90】.”

- **Third-Party KYC APIs:** Services like Onfido, IDfy, Signzy offer end-to-end KYC (document and face verification). They provide SDKs and compliance coverage. *Pros:* Outsources all checks, quick integration. *Cons:* Ongoing cost per user, data goes to 3rd-party, need contracts. This approach is suitable for production scaling, but for an MVP we can simulate or choose open APIs.
  For example, some fintechs use Onfido (now part of Entrust). A blog lists Onfido and others as top ID-verification APIs【30†L6-L14】. We will design our system to allow swapping in these services if needed.

User Experience (UX) Copy Examples

Readiness/Preview Page:

Welcome to [App Name]! To open your account, we’ll need:
- **PAN Card** (uploaded image or via DigiLocker)
- **Aadhaar Card** (linked to your mobile for OTP)
- **Bank Account Details** (cancelled cheque for e-mandate)

The process is **100% online** and takes about **5 minutes**. Make sure you have your mobile phone (for OTPs) handy. Click **Start KYC** to begin.

Other UI tips:

- Show a step indicator (e.g. “Step 2 of 6: Upload PAN”).
- Use clear buttons (“Upload Aadhaar”, “Fetch from DigiLocker”, “Send OTP”).
- Use simple success/error messages (“We could not read your PAN; please upload a clearer image”).

Implementation Checklist & Milestones

Phase 1: User Accounts & Auth

- [ ] Setup project repositories (frontend, backend) and initial boilerplate.
- [ ] Design database schema (users, profiles) and implement user registration endpoint.
- [ ] Implement email/mobile verification (OTP or email link).
- [ ] Develop login endpoint issuing JWT.
- [ ] Frontend: Registration and login forms. Store JWT on login.

Phase 2: Profile and KYC Forms

- [ ] Create user profile API and DB collection (name, address, etc.).
- [ ] Frontend: Profile form to collect personal details. Save to backend.
- [ ] Design KYC request model (status, doc list) in DB.

Phase 3: Document Upload & OCR

- [ ] Implement document upload endpoint: accept file, save to S3, create doc record.
- [ ] Integrate AWS Textract or Google Vision for OCR on upload.
- [ ] Parse OCR results, return to user for confirmation.
- [ ] Frontend: Show OCR-extracted fields, allow edit.
- [ ] Implement storing confirmed KYC fields to `kyc_requests`.

Phase 4: DigiLocker Integration

- [ ] Register as DigiLocker Partner and get API credentials (requires approval).
- [ ] Implement DigiLocker OAuth flow (backend endpoints to initiate and handle callback).
- [ ] Use DigiLocker Issuer APIs to fetch Aadhaar/PAN (require user OTP).
- [ ] Store returned documents (S3) and parse them.
- [ ] Frontend: Add “Login with DigiLocker” button and handle flow.

Phase 5: Aadhaar eSign (Digital Signature)

- [ ] Integrate UIDAI eSign API (register as eSign user).
- [ ] Provide UI for user to enter Aadhaar and OTP, then call backend to get signed AOF.
- [ ] Store signed AOF (PDF) in S3 and log success in KYC record.

Phase 6: Face Capture & Liveness

- [ ] Implement selfie capture on frontend (use getUserMedia).
- [ ] Frontend sends image to backend.
- [ ] Backend calls Rekognition or Face API to compare with Aadhaar photo.
- [ ] Mark pass/fail and store result (e.g. `faceMatch = true/false` in KYC record).
- [ ] Liveness: implement blink or movement test via JS (for simplicity, may use a short video with eyes-open prompt).

Phase 7: Admin Dashboard

- [ ] Create Admin service with authentication (role-based).
- [ ] API to list pending KYC requests with filters (status = Pending).
- [ ] API for approve/reject actions (with optional remarks).
- [ ] Frontend: admin views (React or simple UI) to review docs/images and click Approve/Reject.

Phase 8: Notifications & Audit

- [ ] Hook Postgres triggers or service events: when KYC status changes, call Notification service.
- [ ] Use AWS SNS/SES or Twilio to send real SMS/email to user.
- [ ] Implement Audit logging: every API should write a log entry (e.g. to `audit_logs` collection) via Audit Service.
- [ ] (Optional) Email the signed account form AOF to user for record.

Phase 9: Final Dashboard & Wrapping Up

- [ ] Frontend: Create user dashboard or welcome page after KYC approval.

- [ ] Show link to start investing, and summary of their account ID.
- [ ] Deploy all services to production environment (e.g. AWS EKS/ECS) via CI/CD.

Testing Strategy:

- *Unit Tests:* For each service, write unit tests (Jest/Mocha for Node, PyTest for Python). Cover key logic (e.g. password hashing, OCR parsing).
- *Integration Tests:* Use a tool like Postman/Newman or pytest to test API endpoints (with a test DB). Ensure flows work end-to-end (registration, login, document upload, etc.).
- *UI Tests:* If time, use Cypress or Selenium to automate the main user flows.
- *Security Tests:* Use static code analysis (npm audit, bandit) and OWASP ZAP for dynamic scanning.
- *Performance Tests:* Simulate load (e.g. using Locust) on registration and OCR endpoints to ensure the system scales.
- *Acceptance Criteria:* Every feature above must have at least one passing test case; critical flows must handle invalid input gracefully (e.g. wrong password gives 401, invalid document returns error message).

Milestones:
1. **MVP Ready (Weeks 1-3):** Registration/Login, Profile form, Manual PAN/Aadhaar upload, basic OCR. No DigiLocker yet, admin can review in DB.
2. **Complete KYC Flows (Weeks 4-5):** DigiLocker integration, Aadhaar eSign, face matching. Admin UI.
3. **Polish & DevOps (Weeks 6-7):** Notifications, Audit logging, CI/CD pipeline, SSL, monitoring. Final testing and bug fixes.
4. **Deployment (Week 8):** Go live on AWS, documentation, training for support team.

Acceptance Criteria

- **Functional:** A new user can create an account, go through KYC steps, and either receive “Approved” or “Rejected” with reason. JWT auth must protect endpoints. Admin can approve KYC.
- **Performance:** The system should handle at least X simultaneous users (estimate your scale). Registration and KYC should complete (user action to approved) within ~5 minutes for normal cases.
- **Security:** All sensitive flows (login, uploads) must be over HTTPS. No secrets in code.
- **Compliance:** Audit logs must record all required events (see above). KYC documents are stored securely. Data retention rules are in place.
- **Usability:** Forms have validation and clear instructions. Error messages guide users to fix issues. The readiness page clearly lists requirements. Even a non-tech user can follow the steps without confusion.

Developer Runbook (Step-by-Step)

1. **Set Up Repos & Env:** Initialize GitHub repos for frontend and backend. Set up Node/Express (or chosen framework). Containerize with Docker (write Dockerfiles).
2. **Database:** Choose your DB (Mongo or Postgres). Provision a test DB (e.g. Mongo Atlas or AWS DocumentDB). Create collections/tables as per schema.
3. **Auth Module:** Implement `/register` and `/login`. Use bcrypt for passwords and `jsonwebtoken` for JWT. Test with Postman.

4. **User Profile:** Add endpoints for GET/POST profile. Hook profile creation after successful registration (user flows into profile).
5. **File Upload:** Use a library like `multer` (Node) to accept multipart uploads. Save files temporarily, then upload to S3 (using AWS SDK). Clean temp file. Save S3 URL to DB.
6. **OCR Integration:** After S3 upload, call Textract: e.g. `Textract.detectDocumentText` on the S3 object. Extract useful fields. For forms (PAN), use `detectDocumentText` or `AnalyzeDocument` APIs. Parse JSON response, map to fields. Return to frontend for review.
7. **Face Recognition:** Get API credentials for AWS Rekognition. Store the user's Aadhaar photo (from OCR or DigiLocker) as the baseline. When selfie upload comes, call `compareFaces`. If confidence > threshold, mark match. Do a quick liveness (you can prompt user to blink or hold camera).
8. **DigiLocker:** Register on DigiLocker portal. Implement backend endpoints to start OAuth (get request token), handle redirect URI (get verifier), exchange for access token【1†L101-L110】. Use the token to call `issuer/documents` and `issuer/download` APIs. Save received docs to S3. Use Textract on the downloaded PDFs.
9. **Aadhaar eSign:** Obtain eSign credentials from NSDL or eMudhra. Implement API call: send Aadhaar and OTP, retrieve eSign certificate. Save AOF PDF to S3.
10. **Admin Panel:** Create a simple React/Bootstrap admin page or use a UI framework. It calls protected admin APIs to fetch pending KYC. Implement approve/reject buttons that call backend, which update `kyc_requests` status and write audit logs.
11. **Notifications:** Use AWS SNS/SES. On user registration, email OTP; on KYC status change, email/SMS update. Write a Notification Service that wraps SNS.
12. **Audit Logging:** Every service call (especially login, uploads, admin actions) should create an entry in `audit_logs`. Structure: `{ userId, action: "UPLOAD_PAN", timestamp, details: {...} }`. Ensure this is tamper-proof (or only append).
13. **Testing:** Write tests alongside. Manual testing: follow the full flow with a test user. Try edge cases (expired OTP, bad image, duplicate PAN).
14. **DevOps:** Create AWS resources (via Terraform/CloudFormation). Deploy containers (ECS or EKS). Set environment variables (DB URI, JWT secret from AWS Secrets).
15. **Monitoring:** Configure CloudWatch or Prometheus. Make sure logs show up in dashboard. Test scaling by sending multiple requests.

After these steps, the MVP should be functional. Additional polish (UI styling, error handling, documentation) will ensure production readiness.

Sources: We used official Aadhaar e-KYC documentation【9†L89-L97】, developer guides (DigiLocker integration【1†L101-L110】), fintech UX analysis (Groww steps【14†L70-L78】【14†L110-L119】), AWS architecture best practices【24†L52-L61】【27†L1-L4】, and compliance/audit trail guidelines【20†L50-L58】【21†L153-L161】 to inform this design. Each part above draws on industry standards and examples from leading fintech platforms.
